```
    public async Task PingRequest_ReturnsPong()
    {
        HttpClient client = _fixture.CreateClient();                ❸

        var response = await client.GetAsync("/ping");              ❹

        response.EnsureSuccessStatusCode();                         ❹
        var content = await response.Content.ReadAsStringAsync();   ❹
        Assert.Equal("pong", content);                              ❹
    }
}
```

❶ Implementing the interface allows sharing an instance across tests.

❷ Injects an instance of WebApplicationFactory<T>, where T is a class in your app

❸ Creates an HttpClient that sends requests to the in-memory TestServer

❹ Makes requests and verifies the response as before

One of the advantages of using `WebApplicationFactory` as shown in listing 36.8 is that it requires less manual configuration than using the `TestServer` directly, as shown in listing 36.13, despite performing *more* configuration behind the scenes. The `WebApplicationFactory` tests your app using the configuration defined in your Program.cs and Startup.cs files.

NOTE The generic `WebApplicationFactory<T>` must reference a public class in your app project. It's common to use the `Program` or `Startup` class. If you're using top-level statements for your app (the default in .NET 7), the automatically generated `Program` class is `internal` by default. To make it public and thereby expose it to your test project, add the following partial class definition to your app: `public partial class Program {}`.

Listings 36.8 and 36.7 are conceptually quite different too. Listing 36.7 tests that the `StatusMiddleware` behaves as expected in the context of a